

Edge cases (a request that fails validation) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All versions below assume your `sqlite_persistence` server (or its Advanced version) is already running on `localhost:8000`. Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to check the same thing, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way, with `curl`

```
$ sqlite3 runs.db "SELECT COUNT(*) FROM runs;"
0

$ curl -i -X POST localhost:8000/chat -H "Content-Type: application/json" -d '{}'
```

HTTP/1.1 422 Unprocessable Entity

...

```
{"detail":[{"type":"missing","loc":["body","message"],"msg":"Field required", ...}]}
```

```
$ sqlite3 runs.db "SELECT COUNT(*) FROM runs;"
0
```

The row count before and after is identical — `0` both times (or whatever it already was). This confirms the requirement directly, with no extra code, using tools you already have.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a scripted check with `requests` and real assertions

```
import sqlite3
import requests

conn = sqlite3.connect("runs.db")

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()

response = requests.post("http://localhost:8000/chat", json={})
assert response.status_code == 422, f"expected 422, got {response.status_code}"

after = row_count()
assert before == after, f"row count changed: {before} -> {after}"

print("422 confirmed, no row written")
print(response.json())
```

Expected output:

```
422 confirmed, no row written
{'detail': [{'type': 'missing', 'loc': ['body', 'message'], 'msg': 'Field require
```

Approach 2 — a pytest test, using `TestClient` instead of a running server

```
# test_validation.py
import sqlite3
from fastapi.testclient import TestClient
from main import app, conn

client = TestClient(app)

def row_count() -> int:
```

```
        return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

def test_missing_field_returns_422_and_writes_nothing():
    before = row_count()
    response = client.post("/chat", json={})
    assert response.status_code == 422
    assert row_count() == before
```

Expected output (`pytest test_validation.py`):

1 passed

Difference from Basic: both Intermediate approaches turn a one-time manual check into a real, repeatable assertion instead of eyeballing two `curl` outputs — the kind of check that belongs in a test suite, not just a terminal session. Approach 2 additionally uses `TestClient`, so the check runs in-process with no server or network involved, making it fast enough to run on every change.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a second bad-input shape: wrong type, not missing

```
import sqlite3
import requests

conn = sqlite3.connect("runs.db")

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()

# Case 1: required field missing entirely
r1 = requests.post("http://localhost:8000/chat", json={})
assert r1.status_code == 422
assert row_count() == before

# Case 2: field present, but the wrong type
r2 = requests.post("http://localhost:8000/chat", json={"message": 123})
assert r2.status_code == 422
assert row_count() == before

print("both bad-input cases confirmed: 422, zero rows written")
```

Expected output:

```
both bad-input cases confirmed: 422, zero rows written
```

Pydantic checks both *presence* and *type* — a `message` that's an `int` instead of a `str` fails validation exactly the same way a missing one does, before your route body ever runs.

Approach 2 — proving a multi-step write can't leave a partial row, using a real transaction

This targets the case FastAPI's automatic validation *can't* catch: a request that's perfectly valid, but where your own logic fails partway through more than one database write.

```

import sqlite3
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.row_factory = sqlite3.Row
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        status TEXT
    )
""")

class ChatRequest(BaseModel):
    message: str

def run_chat_two_step(message: str) -> str:
    if message == "fail-after-insert":
        raise RuntimeError("simulated failure between steps")
    return "You said: " + message

@app.post("/chat")
def chat(request: ChatRequest):
    cursor = conn.cursor()
    try:
        cursor.execute("BEGIN")
        cursor.execute(
            "INSERT INTO runs (input, output, status) VALUES (?, ?, ?)",
            (request.message, None, "started"),
        )
        reply = run_chat_two_step(request.message) # can raise partway through
        cursor.execute(
            "UPDATE runs SET output = ?, status = 'completed' WHERE id = ?",
            (reply, cursor.lastrowid),
        )
        conn.commit()
        return {"reply": reply}
    except Exception:
        conn.rollback()
        raise HTTPException(status_code=500, detail="Something went wrong.")

```

```

# test_transaction.py
import sqlite3

```

```
import requests

conn = sqlite3.connect("runs.db")

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()
response = requests.post("http://localhost:8000/chat", json={"message": "fail-aft
assert response.status_code == 500

after = row_count()
assert before == after, f"a partial row was left behind: {before} -> {after}"
print("transaction rollback confirmed: zero rows left in 'started' status")
```

Expected output:

```
transaction rollback confirmed: zero rows left in 'started' status
```

Without `BEGIN / rollback()`, the `INSERT` (status `"started"`) would have been committed on its own before `run_chat_two_step()` ever raised, leaving a row permanently stuck at `"started"` — a real, silent partial write that no amount of 422 handling would have caught, because the request itself was completely valid.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves the one case FastAPI already handles automatically (a malformed request never reaches your database). Approach 1 extends that same proof to a second bad-input shape (wrong type) most people forget to check on purpose. Approach 2 tackles a genuinely different failure mode — not bad input at all, but a *valid* request whose own multi-step logic fails partway through — and shows the actual fix (`BEGIN / commit() / rollback()`), which is a database transaction, not anything FastAPI's request validation could ever provide.

Which one should you actually write? Intermediate Approach 1 or 2 is the right amount of testing for a single-write route like this document's exercises — confirm the required-field 422 case, in a real assertion, and move on. Add Approach 1's wrong-type case any time a field's type actually matters to your logic (a `message: str` that silently became `"123"` behaves very differently from one that's genuinely the string `"123"`). Reach for Approach 2's transaction pattern the moment a single request needs more than one database write to succeed together — which is exactly what the Build Task's `Run` table (status moving from `"started"` to `"completed"` or `"failed"`) needs.